

6. 합성곱 신경망 (3)

goblurry · 약 19시간 전

통계 수정 삭제

딥러닝



딥러닝 파이토치 교과서

▼ 목록 보기

4/4



딥러닝 파이토치 교과서 6장 '합성곱 신경망 2'의 6.1.5-6.3절의 내용을 정리한 포스트입니다.

1. 이미지 분류를 위한 신경망
2. 객체 인식을 위한 신경망
3. 이미지 분할을 위한 신경망

1. 이미지 분류:
 - 1-5. ResNe
2. 객체 인식:
 - 2-1. R-CNN (CNN)
 - 2-2. 공간 피
 - 2-3. Fast R
 - 2-4. Faster
3. 이미지 분할:
 - 3-1. 완전 합
 - 3-2. 합성곱
 - 3-3. U-Net
 - 3-4. PSPN
 - 3-5. DeepI

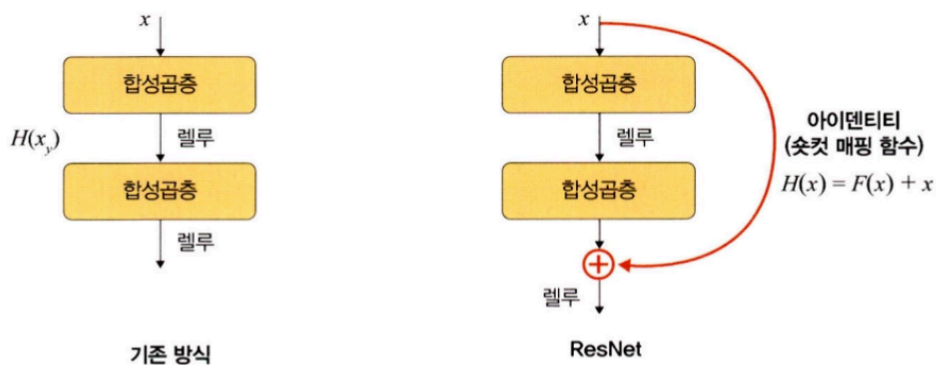
1. 이미지 분류를 위한 신경망

1-5. ResNet

깊어진 신경망을 효과적으로 학습하기 위한 방법으로 **Residual 레지듀얼** 개념을 고안한 네트워크

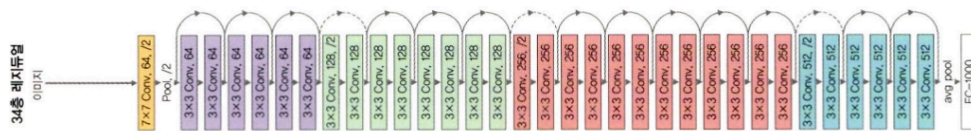
신경망 깊이가 깊어질수록 딥러닝 성능이 좋아질 것 같지만, 실제로는 일정 단계에 다다르면 오히려 성능이 저하된다. 이를 해결하기 위해 **레지듀얼 블록**을 도입하였는데, 이는 기울기가 잘 전파되도록 일종의 숏컷을 만드는 것이다.

▼ 그림 6-25 ResNet 구조



ResNet은 층이 총 152개로 구성되어(GoogLeNet: 22개) 기울기 소멸 문제가 발생할 수 있다. 이에 대한 방지책으로 숏컷을 둔다.

▼ 그림 6-26 ResNet 모델 전체 네트워크

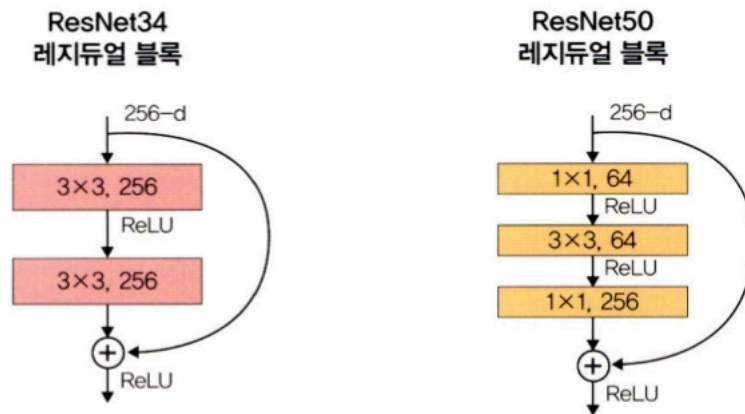


이는 ResNet 모델의 전체 네트워크이다. 보라색, 노란색 등으로 블록이 구분되어 있고, 이렇게 묶인 계층들을 하나의 레지듀얼 블록이라고 한다. 이 레지듀얼 블록을 여러 개 쌓은 것이 ResNet이다.

- 블록: 계층의 묶음. 합성곱층을 하나의 블록으로 묶은 것.

이렇게 계층을 계속 쌓아 늘리면 파라미터 개수에 문제가 생긴다. 계층이 깊어질수록 파라미터는 증가하기 때문이다. 이를 해결하기 위해 병목 블록 bottleneck block을 사용한다.

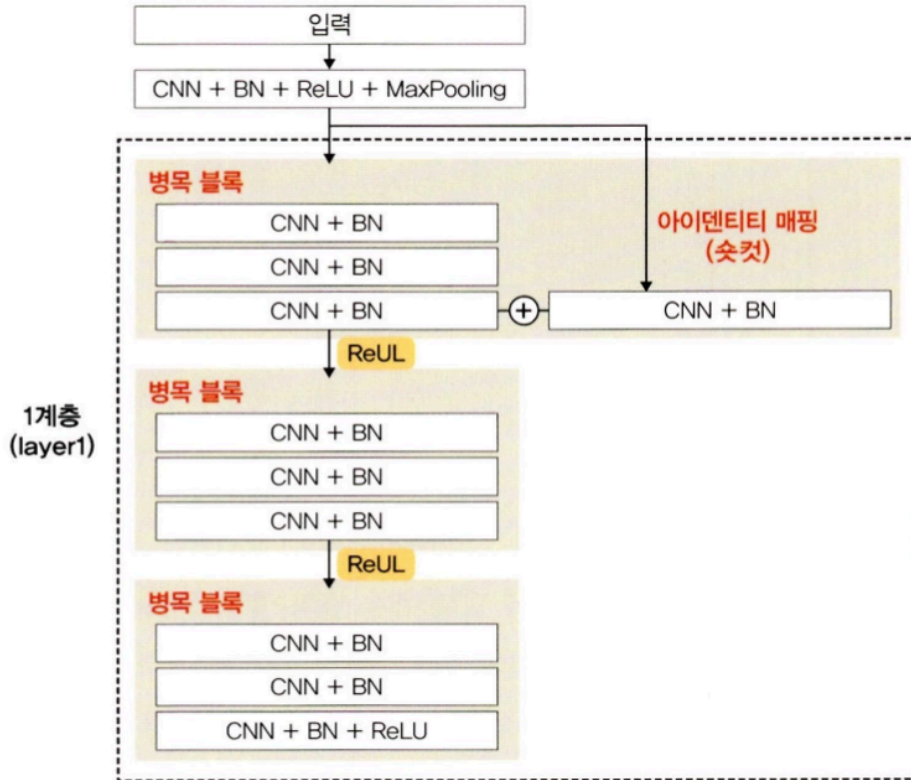
▼ 그림 6-27 기본 블록과 병목 블록



좌측의 ResNet34는 기본 블록을, 우측의 ResNet50은 병목 블록을 사용한다. 파라미터 수를 보면, 기본 블록은 393216M인 반면, 병목 블록은 6,9632M이다. 깊이가 깊어졌는데 오히려 파라미터 수는 감소했다.

이것이 가능한 이유는 병목 블록의 합성곱층의 구조이다. ResNet50에서는 3x3 합성곱층 앞뒤로 1x1 합성곱층이 붙어있다. 이 1x1짜리 층의 채널 개수를 조절하며 차원을 줄였다 늘리는 것이 가능해져 파라미터 수를 줄일 수 있다.

♥ 그림 6-28 아이덴티티 매핑(숏컷)



아이덴티티 매핑: 입력 x 가 어떤 함수를 통과하더라도 다시 x 라는 형태로 출력되게끔 하는 것.

```
def forward(self, x):
    i = x
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.conv2(x)
    x = self.bn2(x)

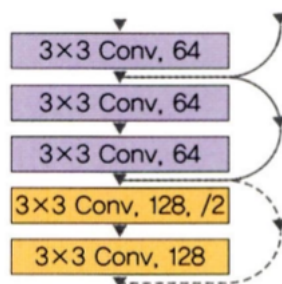
    if self.downsample is not None:
        i = self.downsample(i)

    # identity mapping
    x += i
    x = self.relu(x)

    return x
```

다운샘플: 특성 맵 크기를 줄이기 위한 것으로 풀링과 동일한 역할을 함.

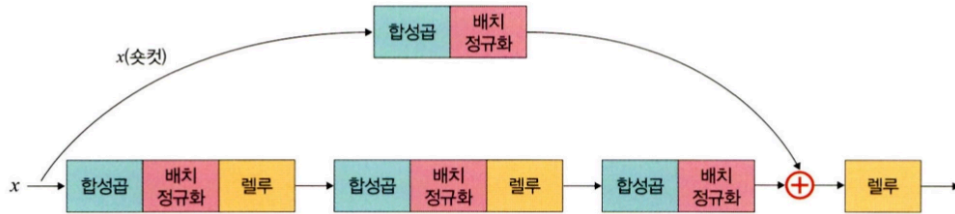
♥ 그림 6-29 ResNet 네트워크의 일부



이러한 구조에서, 보라색 블록의 형상은 (28, 28, 64)이고 노란색은 (14, 14, 128)이다. 즉 형태가 다른데, 이러면 아이덴티티 매핑을 할 수가 없어서 다운샘플이 필요하다.

입출력 형태를 맞추기 위해서는 스트라이드 2를 가진 1x1 합성곱 계층을 한 개 연결하면 된다. 이렇게 입출력 차원이 같은 것을 **아이덴티티 블록**이라고 하며, 반대로 차원이 다르고 입력 차원을 출력 차원에 맞추어 변경해야 하는 것을 **프로젝션 숏컷** 혹은 **합성곱 블록**이라고 한다.

♥ 그림 6-30 합성곱 블록



🌟 ResNet: 기본적으로 VGG19 구조를 뼈대로, 합성곱층을 추가해 깊게 만든 후 숏컷을 추가한 네트워크.

2. 객체 인식을 위한 신경망

객체 인식: 이미지/영상 내에 있는 객체를 식별하는 컴퓨터 비전 기술

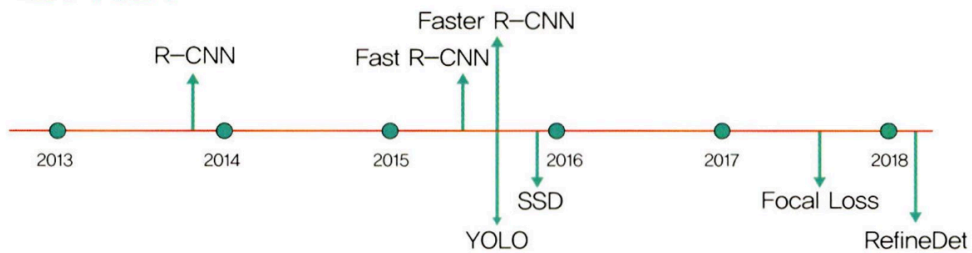
주요 문제: 여러 객체에 대한 분류 + 객체의 위치 정보를 파악하는 위치 검출(localization)

자율 주행 자동차, CCTV, 무인 점포 등에서 활용되는 기술이다.

딥러닝을 사용한 객체 인식 알고리즘은 크게 두 단계로 나눌 수 있다.

♥ 그림 6-35 1단계 객체 인식 vs 2단계 객체 인식 흐름도

2단계 객체 인식



1단계 객체 인식

1단계는 위의 두 문제(분류+위치 검출)를 **동시에** 행하는 방법이고, 2단계는 **순차적**으로 행하는 방법이다.

전자는 비교적 빠르지만 정확도가 낮고, 후자는 그 반대이다.

교재에서는 2단계 객체 인식 알고리즘을 다룬다. 대표적 알고리즘로는 CNN을 처음으로 적용시킨 R-CNN 계열이 있다.

2-1. R-CNN (Region Based CNN)

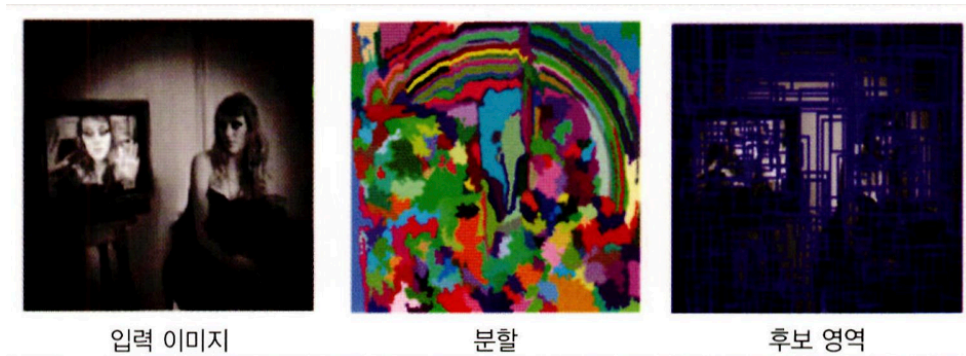
앞서 공부한 CNN(이미지 분류 수행)과 이미지에서 객체가 있을 법한 영역을 제안해 주는 **후보 영역 알고리즘**을 결합한 알고리즘.

이전의 객체 인식 알고리즘: Sliding window 방식 적용. 일정 크기를 가지는 윈도우를 가지고 이미지의 모든 영역을 탐색하면서 객체를 검출해 내는 방식. 비효율적.

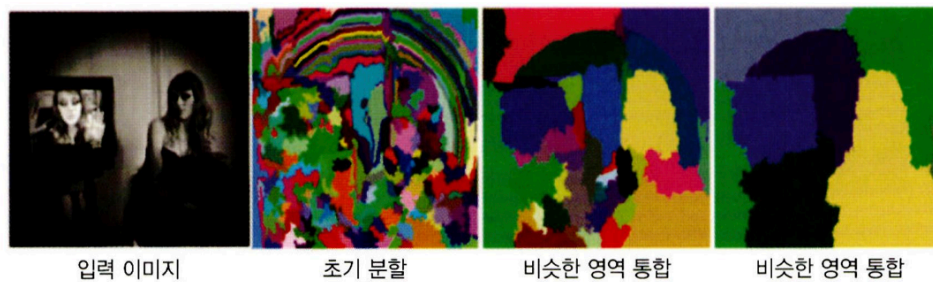
후보 영역 알고리즘: Region proposal. 선택적 탐색 selective search 방식을 이용해, 영상/이미지에서 객체가 있을 법한 영역을 찾는 방식.

선택적 탐색 : 객체 인식 및 검출을 위한 가능한 후보 영역을 알아내는 방법으로, 분할 방식을 사용하여 seed를 설정하고, 그 시드에 대해 완전 탐색을 적용한다. 다음 세 단계를 거친다. (시드: 영상에서 특정 기준점이 되는 픽셀. 시드에서부터 점점 의미가 같은 영상 범위까지 픽셀을 확장하며 분할함.)

1. 초기 영역 생성: 입력 이미지를 여러 개의 영역으로 분할하여 각각의 객체가 영역 한 개에 할당될 수 있도록 초기 영역을 생성한다.



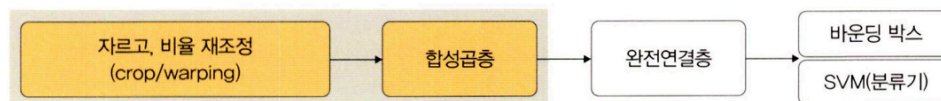
2. 작은 영역의 통합: 1단계에서 나눈 것들을 비슷한 영역으로 통합한다.



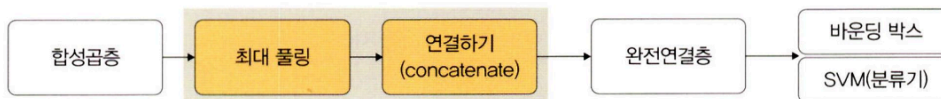
3. 후보 영역 생성: 2단계에서 통합된 이미지들을 기반으로 후보 영역(바운딩 박스)을 추출한다.

♥ 그림 6-40 공간 피라미드 풀링

R-CNN

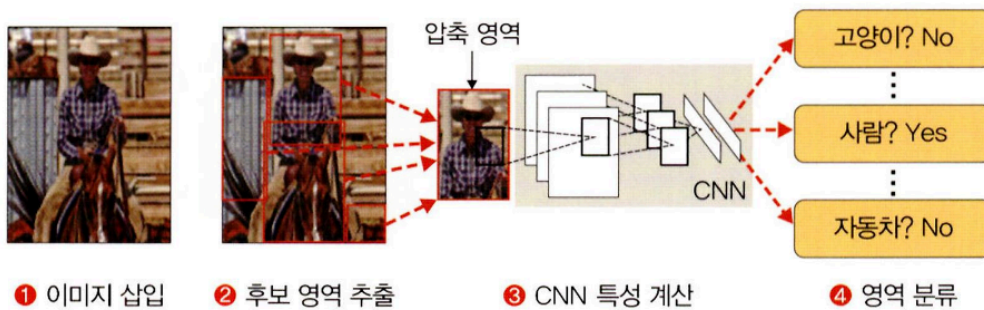


공간 피라미드 풀링



공간 피라미드 풀링층

학습 절차



1. 입력 이미지를 받는다.
2. 2000개의 bounding box를 선택적 탐색 알고리즘으로 추출 후 잘라내고, CNN 모델에 넣기 위해 같은 크기(227px*227px)로 통일한다.
3. 크기가 동일한 이미지 2000개에 각각 CNN 모델을 적용한다.
4. 각각 분류를 진행하여 결과를 도출한다.

이러한 R-CNN은 성능이 뛰어나지만, 아래와 같은 단점으로 크게 발전하지 못했다.

- 복잡한 학습 과정
- 긴 학습 시간과 대용량 저장 공간
- 객체 검출 속도 문제

이에 대한 해결책으로 등장한 것이 **Fast R-CNN**(2-3)이다.

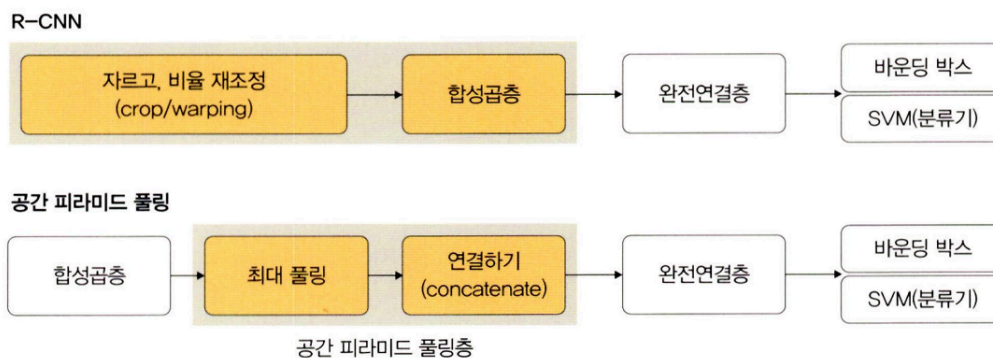
2-2. 공간 피라미드 풀링

기존 CNN 구조: 완전연결층을 위해 입력 이미지 고정 필요 → 고정된 크기로 자르거나 비율 조정.

문제점: 물체의 일부분이 잘리거나 본래 생김새와 달라지게 됨.

🌟 해결책: 공간 피라미드 풀링 Spatial pyramid pooling

▼ 그림 6-40 공간 피라미드 풀링



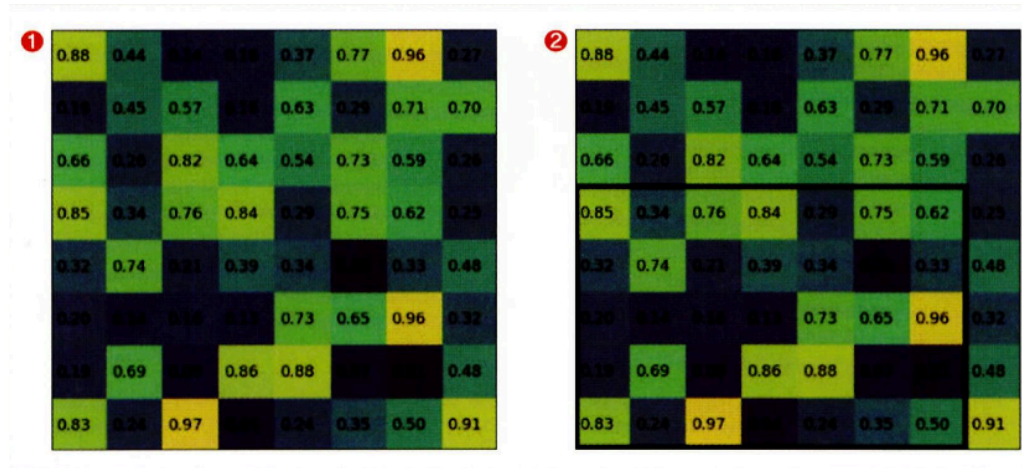
R-CNN과 비교하면, 합성곱층 진입 전에 입력 이미지를 조정하는 과정이 없다. 즉, 공간 피라미드 풀링은 입력 이미지의 크기에 관계없이 합성곱층을 통과시키고, **완전 연결층에 전달되기 전에 특성 맵들을 동일 크기로 조절하는 풀링층을 적용하는 기법**이다.

그 결과, 원본 이미지의 특징이 훼손되지 않는 특성 맵을 얻을 수 있고, 이미지 분류나 객체 인식 등 여러 작업에 적용할 수 있게 된다.

2-3. Fast R-CNN

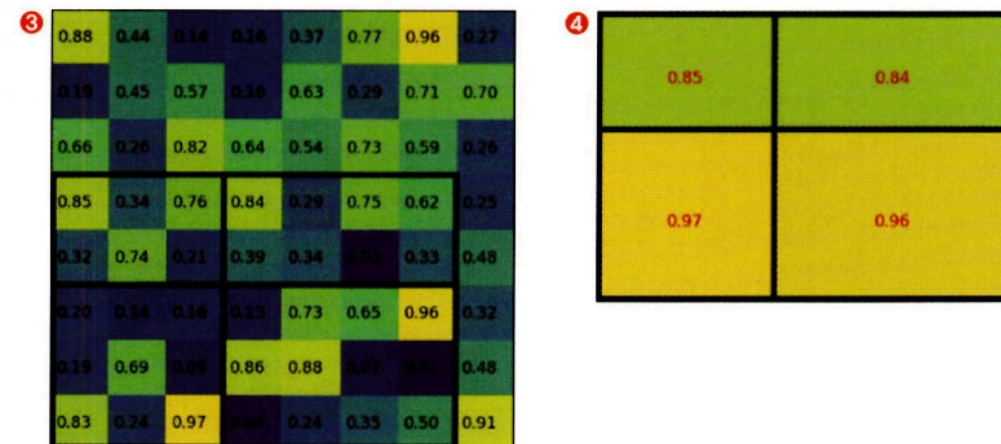
R-CNN은 입력 이미지를 받아서 바운딩 박스를 추출해 잘라내고 매 박스마다 CNN 모델을 돌려 분류했다. 이 과정에서 소요되는 학습 시간이 길다는 문제가 있었는데, Fast R-CNN은 이 문제를 해결하기 위해 **RoI 풀링**을 도입했다.

+) RoI 풀링: 크기가 다른 특성 맵의 영역마다 스트라이드를 다르게 최대 풀링을 적용하여 결과값의 크기를 동일하게 맞추는 방법



1. 8x8 특성 맵

2. 1에서 선택적 탐색으로 뽑아낸 7x5 후보 영역

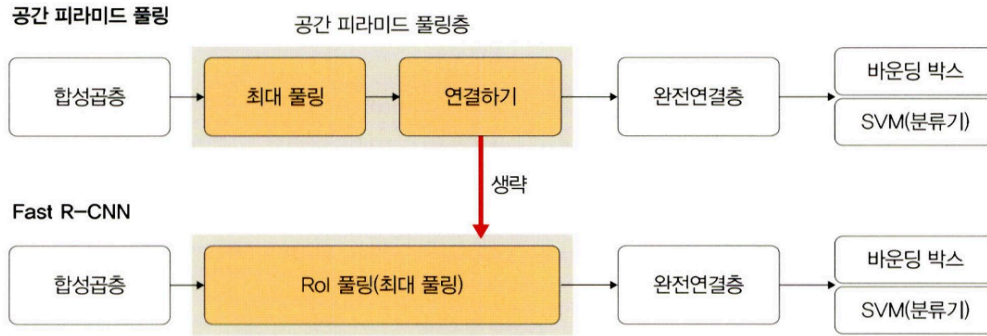


3. 스트라이드(7/2=3, 5/2=2)로 풀링 영역 지정

4. 최대 풀링 적용하여 2x2 결과 획득

선택적 탐색 과정에서 찾은 바운딩 박스가 모델을 통과하면서 유지되도록 하고, 최종 CNN 특성 맵은 풀링을 적용하여 완전연결층을 통과하도록 크기를 조정한다.

♥ 그림 6-41 Fast R-CNN



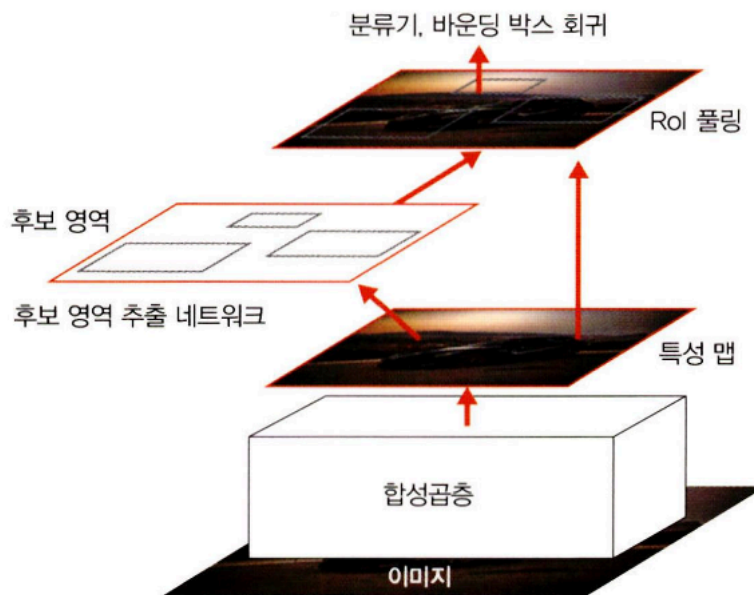
2-4. Faster R-CNN

후보 영역 생성을 CNN 내부 네트워크에서 진행하는 방향으로 수정하여 설계된 객체 인식 네트워크

✨ 기존 Fast R-CNN + 후보 영역 추출 네트워크 (Region Proposal Network, RPN)

후보 영역을 생성하는 과정은 속도가 느려 기존 네트워크의 한계가 되었었는데(외부의 느린 선택적 탐색, CPU로 계산) 내부에서 GPU를 활용해 빠르게 계산을 수행한다.

♥ 그림 6-43 Faster R-CNN



마지막 합성곱층 다음에 위치하고, 그 뒤에 RoI 풀링과 분류기, 바운딩 박스 회귀(바운딩 박스가 이미지 객체를 정확히 포착하여 포함시킬 수 있도록 조정)가 위치한다.

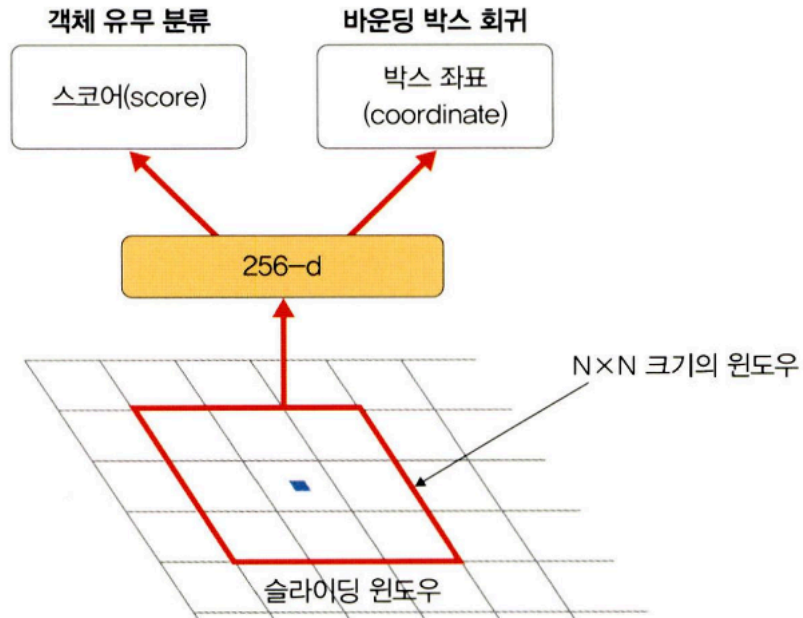
후보 영역 추출 네트워크

입력: 특성 맵 NxN 크기의 작은 윈도우 영역

해당 영역에 객체가 존재하는지를 판단하기 위해 이진 분류를 수행하는 작은 네트워크를 생성한다. 바운딩 박스도 위치 보정을 위해 추가한다.

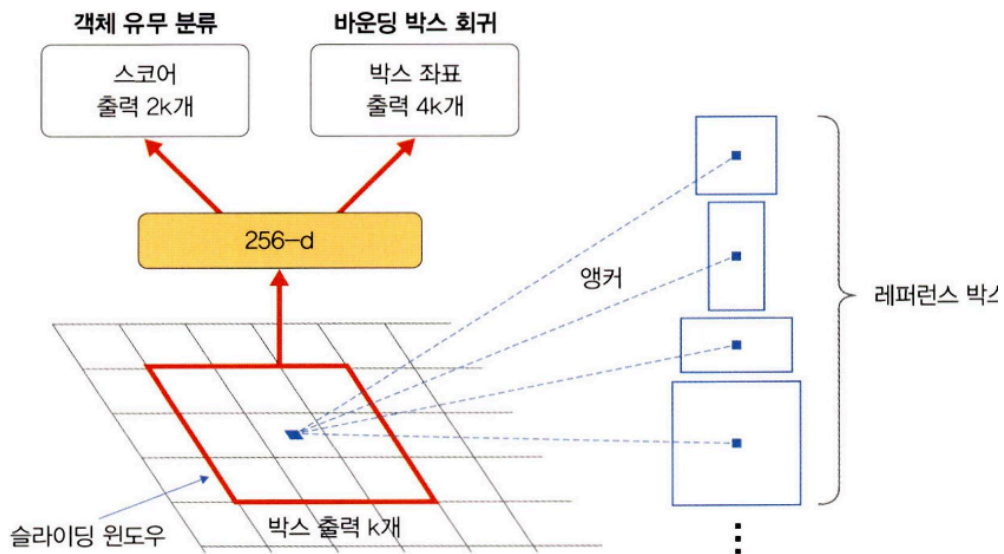
슬라이딩 윈도우 방식으로 설계해 둔 NxN 크기의 작은 윈도우 영역을 이용해 객체를 탐색하고, 이를 통해 하나의 특성 맵에서 모든 영역에 대한 객체 존재 유무를 확인한다.

▼ 그림 6-44 후보 영역 추출 네트워크



그러나 후보 영역 추출 네트워크는 이미지에 존재하는 객체들의 크기와 비율이 다양하기 때문에 고정된 $N \times N$ 크기의 입력만으로는 여러 이미지를 수용하기 어렵다는 문제가 있다. 이를 보완하기 위해, 여러 크기와 비율의 레퍼런스 박스 k 를 미리 정의하고, 각각의 슬라이딩 윈도우 위치마다 박스 k 개를 출력하도록 하는데 이를 앵커라고 한다.

▼ 그림 6-45 앵커



후보 영역 추출 네트워크의 출력 값: 모든 앵커 위치에 대해 각각 객체와 배경을 판단하는 2k개의 분류에 대한 출력 + x, y, w, h 위치 보정값을 위한 4k개의 회귀 출력

3. 이미지 분할을 위한 신경망

이미지 분할 image segmentation: 신경망을 훈련시켜 이미지를 픽셀 단위로 분할하는 것.

분할된 이미지에 포함된 객체를 추출한다. 대표적 네트워크로 완전 합성곱 네트워크, 합성곱/역합성곱 네트워크, U-Net, PSPNet, DeepLabv3/v3+가 있다.

3-1. 완전 합성곱 네트워크

Fully Convolution Network, 완전연결층을 1x1 합성곱으로 대체하는 네트워크.

기존 완전연결층의 한계: 고정된 크기의 입력만 받아들이고 완전연결층을 거치면 위치 정보가 사라짐.

이미지 분류에서 우수한 성능을 보였던 CNN 기반의 모델(AlexNet, VGG16, GoogLeNet)을 변형시켜 이미지 분할에 적합하게 만든 네트워크이다.

위치 정보가 보존되고 입력 이미지에 대한 크기 제약이 사라진다는 장점이 있다.

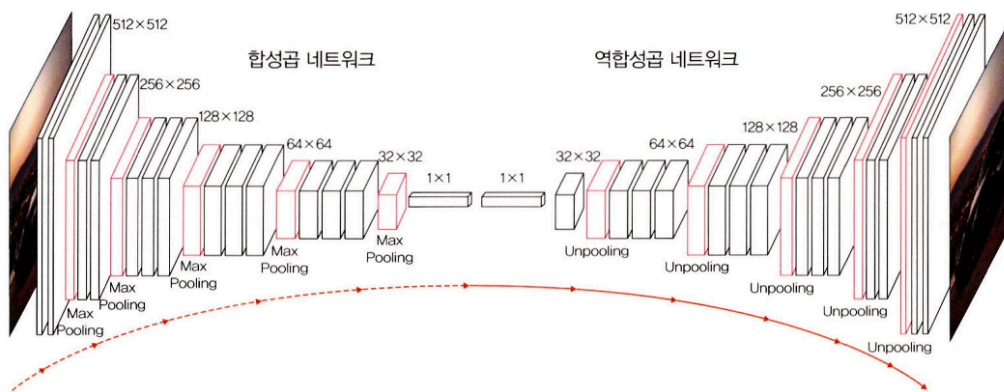
3-2. 합성곱 & 역합성곱 네트워크

FCN은 앞서 말한 장점에도 다음과 같은 단점이 존재한다.

- 여러 단계의 합성곱층과 풀링층을 거치며 **해상도 저하**
- 저하된 해상도를 복원하기 위해 업 샘플링을 사용하여 이미지 세부 정보 손실 (업 샘플링: 최종 이미지 크기가 입력 이미지 크기와 같도록 하는 것)

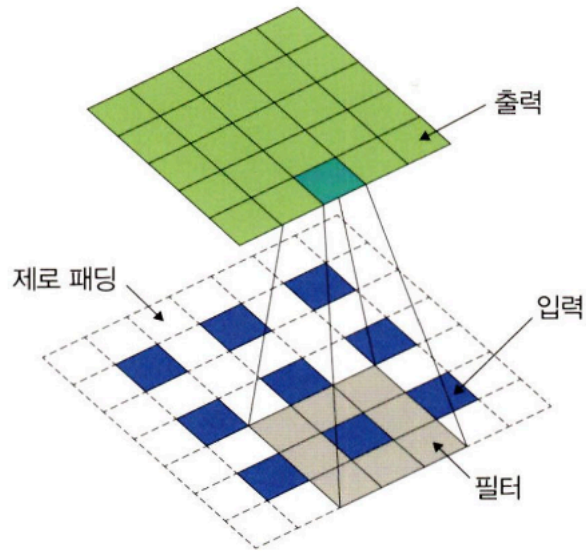
이에 대한 해결책으로 도입된 것이 **합성곱 & 역합성곱** 네트워크이다.

♥ 그림 6-47 합성곱 & 역합성곱 네트워크



CNN에서 합성곱층은 합성곱을 사용해 특성 맵의 크기를 줄였다. 역합성곱층은 이름만 봐도 알 수 있듯, 이와 반대로 작동하여 **특성 맵의 크기를 증가**시킨다. CNN의 최종 출력 결과를 입력 이미지와 같은 크기로 만들고자 할 때 사용한다.

▼ 그림 6-48 역합성곱 진행 방식



입력 이미지의 각각의 픽셀 주위에 **제로 패딩**을 추가한 뒤, 합성곱 연산을 수행한다.

3-3. U-Net

U-Net은 바이오 메디컬 이미지 분할을 위한 합성곱 신경망이다.

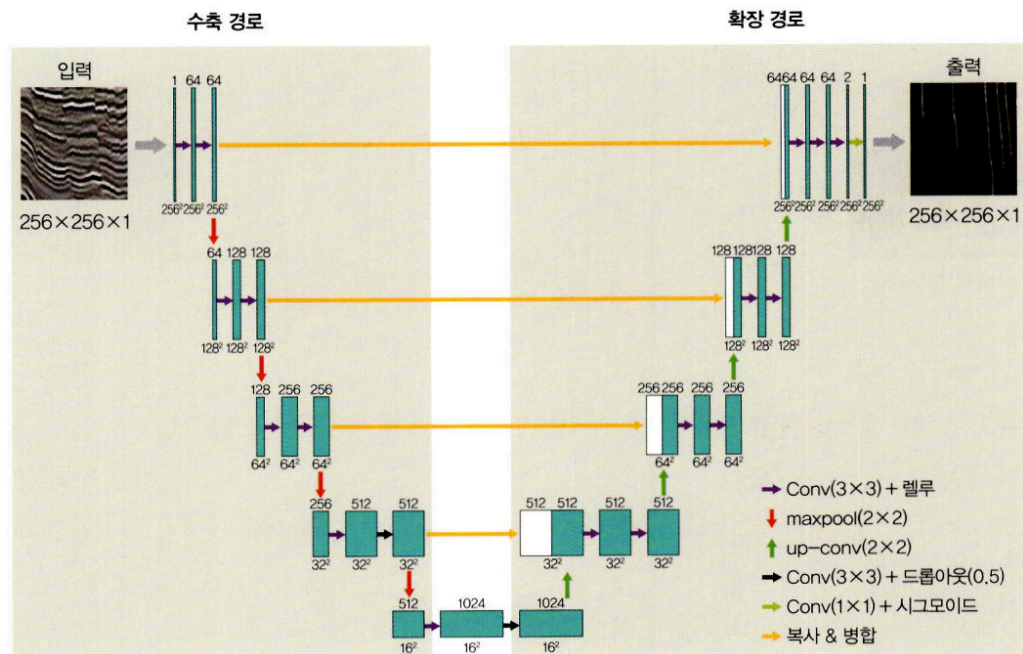
[주요 특징]

1. 빠른 속도: 이미 검증이 끝난 패치(이미지 인식 단위)는 건너뛴. 기존의 슬라이딩 윈도우 방식은 이전 패치에서 검증이 끝난 부분을 다음 패치에서 또 검증하여 속도가 느림.
2. 지역화 트레이드 오프에 빠지지 않음: 일반적으로 패치 크기가 클 때에는 넓은 범위의 이미지 인식에 강해 맥락은 잘 파악하지만, 지역화에 한계가 있음. (지역화: 이미지 안에 객체 위치 정보를 출력하는 것) 이 문제를 개선함.

[구조]

- 완전 합성곱 네트워크(FCN)를 기반으로 구축.
- 수축 경로(contracting path): 컨텍스트(맥락) 포착
- 확장 경로(expansive path): 특성 맵을 업 샘플링 하고 수축 경로에서 포착한 특성 맵의 컨텍스트와 결합해 정확한 지역화 수행

♥ 그림 6-51 U-Net



U-Net은 3x3 합성곱이 주를 이루는데, 각각의 합성곱 블록은 3x3 두 개와 그 사이 드롭아웃으로 구성된다. 크기가 다양한 이미지의 객체를 분할하기 위해, 크기가 다양한 특성 맵을 병합할 수 있도록 다운 샘플링(좌)과 업 샘플링(우)을 순서대로 반복하는 구조로 이루어져 있다.

3-4. PSPNet

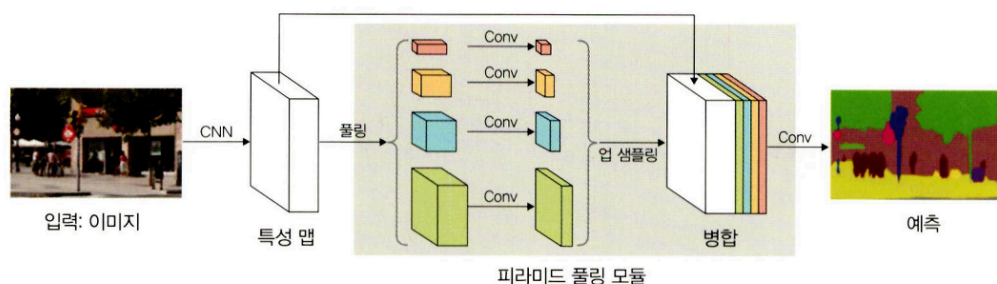
Pyramid Scene Parsing Network: 시맨틱 분할 알고리즘.

완전연결층의 한계를 극복하기 위해 **피라미드 풀링 모듈**을 추가했다.

[훈련 과정]

1. 이미지 출력이 서로 다른 크기가 되도록 여러 차례 풀링을 합니다. 즉, 1×1 , 2×2 , 3×3 , 6×6 크기로 풀링을 수행하는데, 이때 1×1 크기의 특성 맵은 가장 광범위한 정보를 담습니다. 각각 다른 크기의 특성 맵은 서로 다른 영역들의 정보를 담는다고 이해하면 됩니다.
2. 이후 1×1 합성곱을 사용하여 채널 수를 조정합니다. 풀링층 개수를 N 이라고 할 때 출력 채널 수=입력 채널 수/ N 이 됩니다.
3. 이후 모듈의 입력 크기에 맞게 특성 맵을 업 샘플링합니다. 이 과정에서 양선형 보간법(bilinear interpolation)이 사용됩니다.
4. 원래의 특성 맵과 1~3 과정에서 생성한 새로운 특성 맵들을 병합합니다.

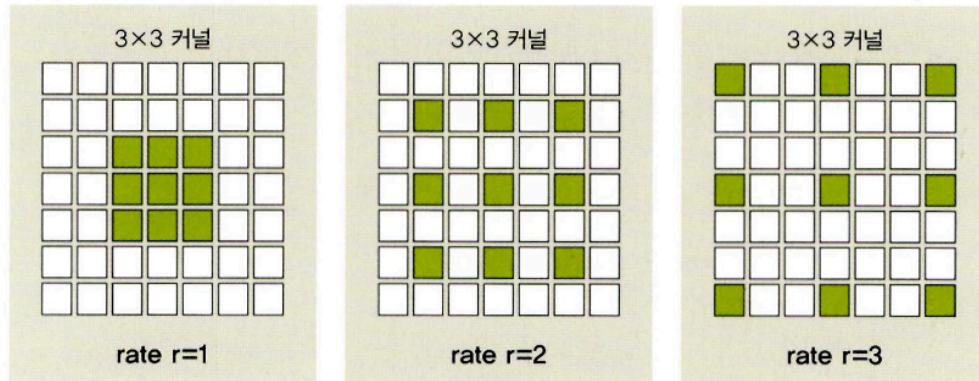
♥ 그림 6-52 PSPNet



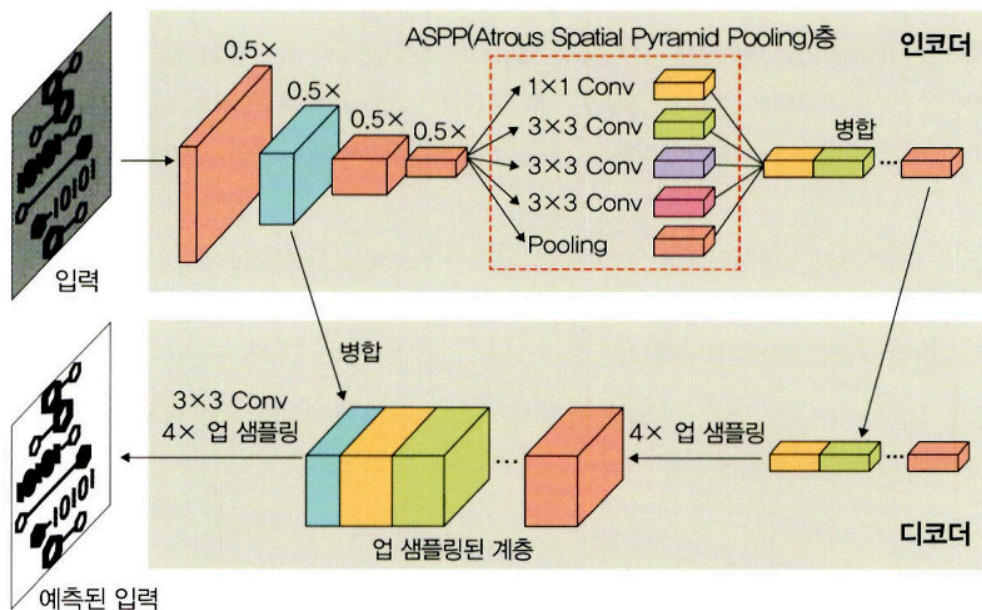
3-5. DeepLabv3 / DeepLabv3+

이 네트워크 역시 완전연결층의 단점 보완을 위한 것으로, **Atrous** 합성곱을 사용한다. 인코더와 디코더 구조를 가지는데, 일반적인 인코더-디코더 구조에서 하지 못하는 인코더에서 추출된 특성 맵의 해상도 제어가 가능하다.

▼ 그림 6-55 Atrous 합성곱



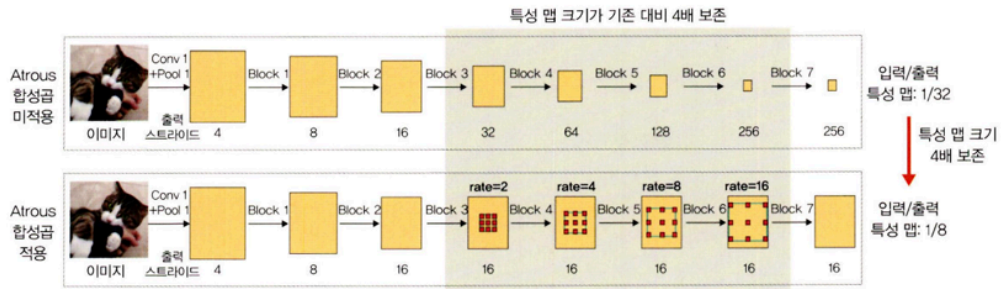
▼ 그림 6-54 DeepLab의 인코더-디코더 구조



위와 같이 Atrous 합성곱은 필터 내부에 빈 공간을 두고 작동하는데, 빈 공간을 얼마나 둘지 결정하는 파라미터로 **rate** 를 사용한다. (값이 커질수록 빈 공간이 늘어난다.)

이 합성곱을 활용하면 파라미터의 개수를 늘리지 않으면서도 **수용 영역**을 크게 키울 수 있다. 수용 영역을 확대하면 **특성을 찾는 범위가 넓어져** 이미지 분할의 성능이 높아진다. (수용 영역: 외부 자국이 전체가 아니라 특정 영역에만 영향을 주는 것.)

♥ 그림 6-56 Atrous 합성곱 효과



goblurry



이전 포스트

5. 합성곱 신경망 (2)

0개의 댓글

댓글을 작성하세요

댓글 작성



Powered by
Stellate